Fuel Ordering & Monitoring Platform

Design Proposal Document

Status: Ready for MVP Development

Table of Contents

Executive Summary

This document outlines the design of a **Fuel Ordering & Monitoring Platform**—an e-commerce–like SaaS solution that connects fuel sellers (oil fields/depots) with fuel clients (petrol stations, industrial customers) via **Transportation Service Providers (TSPs)**.

Key Features

- **Multi-tenant workspace model**: users belong to one or more workspaces with different roles.
- **Seller registration**: oil field/depot owners register, add depot locations, invite Transportation Service Providers (TSPs).
- **TSP registration with KYC**: TSPs accept invitations, submit compliance documents (driver licenses, IDs, permits) for seller manager approval before vehicle operations.
- **Vehicle approval workflow**: Trucks require commercial and safety approval flags to be set to true before sensor integration.
- **Sensor integration tickets**: After truck registration and approvals, TSPs create tickets for platform admin to integrate Galileosky sensors and generate QR codes.
- **Client registration**: clients register and manage multiple delivery addresses.
- **Order lifecycle**: clients place orders → sellers accept and assign to TSP → TSP manually selects trucks and assigns fuel types to compartments → trucks deliver → clients scan QR codes → trucks return to depot.

- **Multi-fuel compartments**: Trucks can have compartments with different fuel types; TSP assigns fuel type **per compartment during order assignment** (not during registration).
- **Real-time telemetry**: IoT devices (via flespi) publish GPS and fuel sensor data to AWS; backend ingests and processes in real-time.
- **Geofencing & QR validation**: deliveries only unlock when truck arrives at destination and client scans the assigned truck's QR code. Truck journey completes when it returns to depot geofence.
- **Event-driven notifications**: system emits events (TSP invitation, KYC approval, ticket creation, integration complete, order assignment, etc.) and sends email notifications.

Target Scale

- ~5–10 seller workspaces.
- ~10–20 TSPs across all workspaces.
- ~50–100 trucks across all TSPs.
- ~100–200 orders/day.
- ~5–10 IoT devices publishing data simultaneously.

Designed to scale to 1000s of trucks and 10k+ daily orders without major refactoring.

System Overview

Stakeholders & Roles

1. **Platform Admin** (Your Company)
    - Create and manage seller workspaces.
    - Handle sensor integration tickets (configure Galileosky devices).
    - Generate QR codes for trucks after successful sensor integration.
    - Validate TSP KYC approval status and vehicle approval flags before sensor integration.
    - Monitor platform health, usage.
    - Configure system-wide settings.
2. **Seller Manager** (Oil Field/Depot Owner)
    - Register and manage their company/workspace.
    - Add depot locations.
    - **Invite Transportation Service Providers (TSPs) via email**.
    - **Review and approve TSP KYC/compliance documents**.
    - **Set vehicle commercial and safety approval flags**.
    - Accept incoming orders and **assign orders to TSPs**.
    - View orders, truck fleet status (across all TSPs), and delivery history.
    - Manage workspace members and their roles.

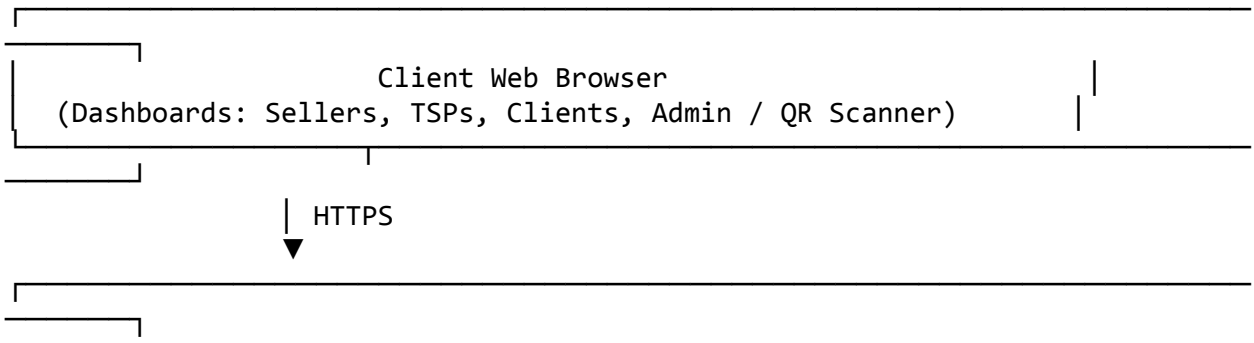
3. **Transport Admin (TSP)** (Transportation Service Provider)
    - Accept invitation from seller manager to join workspace.
    - **Upload and maintain required KYC / compliance documents** (driver licenses, government IDs, permits, etc.).
    - **Register and configure trucks** (vehicle details, compartments, capacities), including declaring commercial and safety status.
    - Create **sensor integration tickets** when truck registration is complete (only if KYC approved).
    - **Manually select trucks and drivers** for orders assigned by seller.
    - **Assign fuel type per compartment during order fulfillment** (not during registration).
    - Monitor truck telemetry and delivery status.
4. **Client** (Petrol Station, Industrial Customer, etc.)
    - Register company and add delivery locations.
    - Place fuel orders specifying location and quantity.
    - View order status and assigned truck details.
    - **Accept deliveries** by scanning truck QR codes at destination.
    - View delivery history.

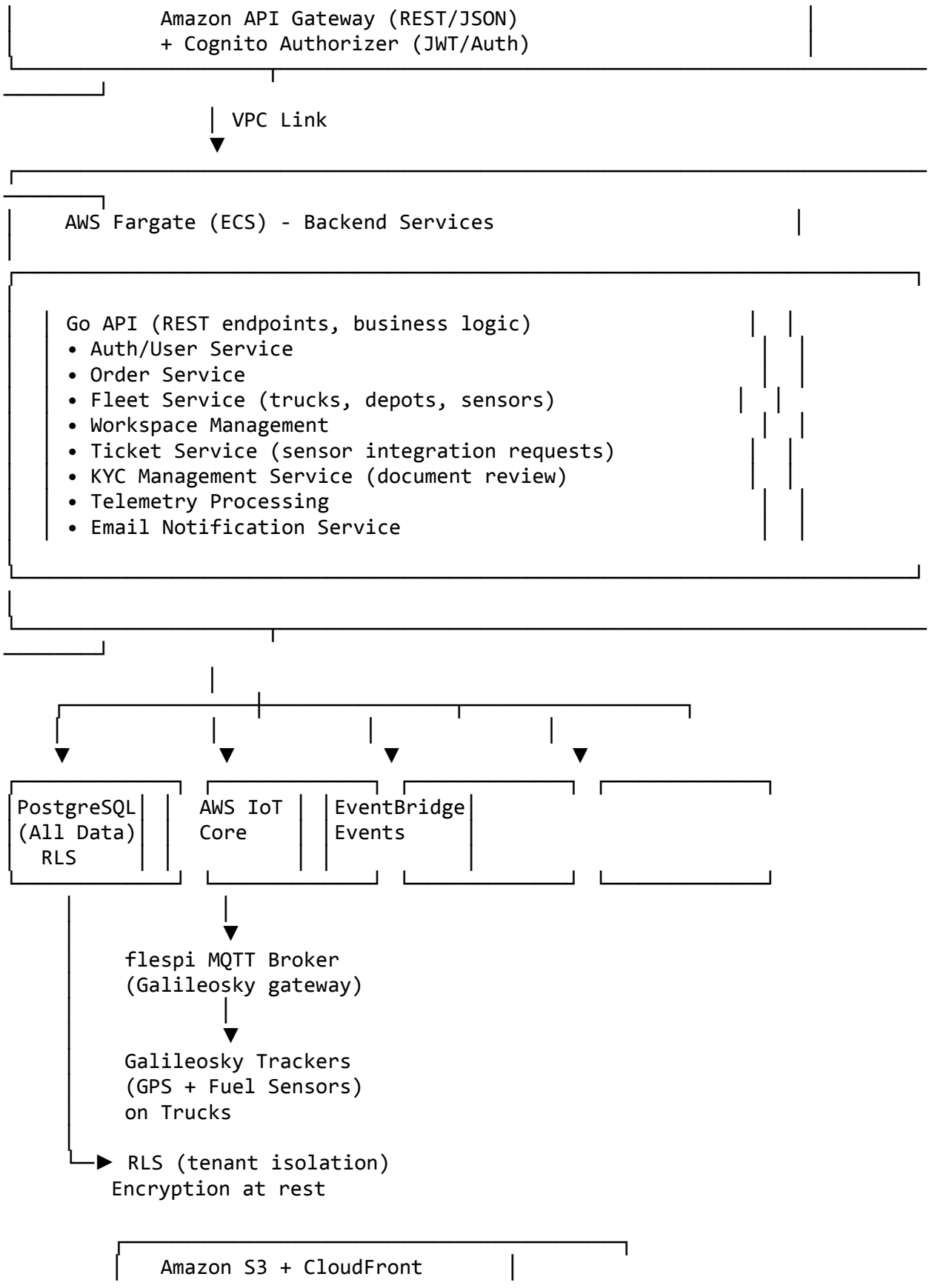
Workspace Model

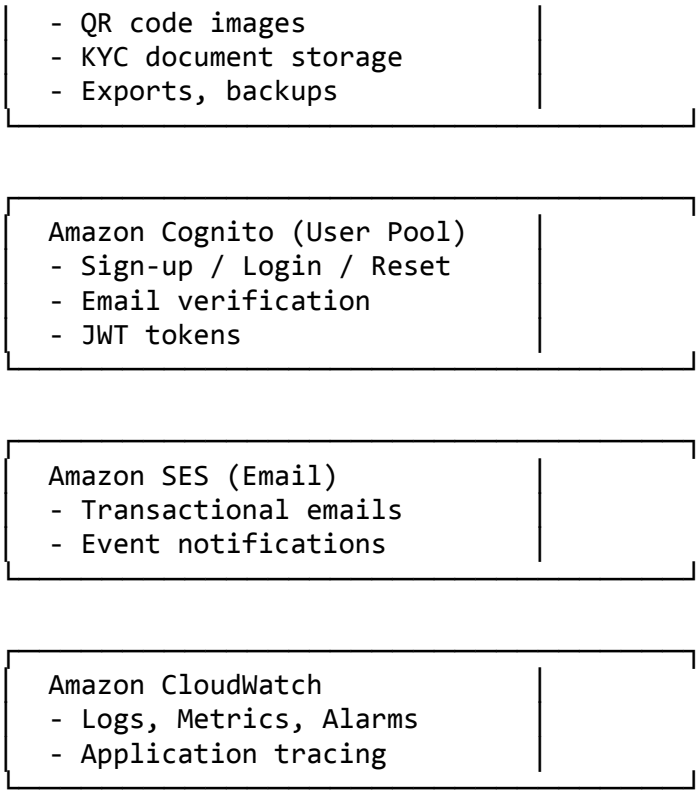
- Users can belong to **multiple workspaces** with **different roles per workspace**.
- Example: John is a `SELLER_MANAGER` at Workspace A, `TRANSPORT_ADMIN` at Workspace B, and `CLIENT` at Workspace C.
- Sellers invite TSPs via email; TSPs join seller's workspace as `TRANSPORT_ADMIN`.
- One TSP can work with **multiple sellers** (multiple workspaces).
- Trucks belong to seller's workspace but are **managed by TSPs**.
- Each workspace is isolated; queries are tenant-scoped via PostgreSQL Row-Level Security (RLS).

Architecture

3.1 High-Level System Diagram




Amazon API Gateway (REST/JSON)
+ Cognito Authorizer (JWT/Auth)
VPC Link
AWS Fargate (ECS) - Backend Services
Go API (REST endpoints, business logic)
• Auth/User Service
• Order Service
• Fleet Service (trucks, depots, sensors)
• Workspace Management
• Ticket Service (sensor integration requests)
• KYC Management Service (document review)
• Telemetry Processing
• Email Notification Service
PostgreSQL (All Data) RLS
AWS IoT Core
EventBridge Events
flespi MQTT Broker
(Galileosky gateway)
Galileosky Trackers
(GPS + Fuel Sensors)
on Trucks
RLS (tenant isolation)
Encryption at rest
Amazon S3 + CloudFront



3.2 Components Overview

Frontend

- **React/Next.js** single-page application.
- Separate dashboards:
  - **Seller Dashboard**: workspace management, TSP invitations, KYC document review, vehicle approvals, order acceptance and TSP assignment, real-time map.
  - **TSP Dashboard**: KYC document upload, truck registration, ticket creation, order assignment (manual truck selection with fuel type assignment), telemetry monitoring.
  - **Client Dashboard**: order placement, order tracking, QR scanner.
  - **Admin Dashboard**: workspace management, ticket management (sensor integration with KYC/approval validation), QR code generation, system config.
- QR scanner: `jsQR` library for in-browser QR detection via WebRTC.
- Responsive design; works on mobile and desktop.

Backend API (Fargate)

- **Technology**: Go (recommended given your backend expertise).
- **Architecture**: modular monolith with internal service modules.
  - `AuthService`: login, registration, JWT, email invites.

  - `WorkspaceService`: workspace CRUD, member management, roles, TSP invitations.
  - `KYCService`: document upload, review workflow, approval status tracking.
  - `OrderService`: order lifecycle, TSP assignment logic.
  - `FleetService`: truck registration, depot management, compartment/sensor config, approval flags, QR code generation.
  - `TicketService`: ticket CRUD, status transitions, prerequisite validation (KYC + approvals), notifications.
  - `TelemetryService`: ingest IoT data, aggregate fuel levels per compartment, update truck state, geofence checks.
  - `NotificationService`: email triggers on key events.
- **API Pattern**: RESTful JSON, authenticated via JWT.
- **Error Handling**: consistent error response format.
- **Logging**: structured JSON logs to CloudWatch.
- **Deployment**: Docker image → Amazon ECR → AWS Fargate.
- **Scaling**: auto-scaling based on CPU/memory via ECS Service.

Data Storage

PostgreSQL (Single Database, All Data)

- Single relational database shared across all tenants/workspaces.
- All tables include `workspace_id` column.
- PostgreSQL Row-Level Security (RLS) enforces tenant isolation at DB level.
- **Telemetry Storage**: `truck_telemetry` table with `truck_id`, `timestamp`, and JSONB `compartment_sensors` field.
- **Ticket Storage**: `tickets` table for sensor integration and future use cases.
- **KYC Storage**: `tsp_kyc_documents` table for compliance document management.
- Benefits: single database, simple operational model, easy to query across entities for development.
- Backup and failover handled by RDS (automatic, multi-AZ).

S3 + CloudFront

- QR code PNG images per truck (generated after sensor integration).
- KYC/compliance document storage (secure, signed URLs).
- Database backups, exported reports.
- CloudFront caching for fast delivery.

IoT Telemetry Ingestion

flespi → AWS IoT Core pathway:

1. Galileosky devices send data via proprietary protocol to **flespi MQTT broker**.
2. flespi allows configuration of integrations/streams.

3. Configure an **AWS IoT Core endpoint** in flespi.
4. flespi bridges MQTT messages to AWS IoT Core.
5. **AWS IoT Rules** subscribe and:
   - Parse telemetry JSON (includes sensor IDs and values).
   - Trigger Lambda to process and store in PostgreSQL.
   - Emit events to EventBridge.

Data flow:

```
Galileosky Device (in truck)
  ↓ (proprietary protocol, ~10-30 sec interval)
  ↓ Sends: sensor_1_id, sensor_1_value, sensor_2_id, sensor_2_value, GPS,
etc.
flespi Gateway
  ↓ (MQTT forward)
AWS IoT Core (topic: galileosky/<device_id>/data)
  ↓ (IoT Rule)
Lambda Function (parse, map sensor_id → compartment, apply calibration)
  ↓
PostgreSQL (TruckTelemetry table per-compartment fuel + Trucks state update)
  ↓
EventBridge (emit TRUCK_POSITION_UPDATED event)
  ↓
Backend event handlers (geofence check, fuel level tracking)
```

Geofencing

Simple Distance-Based Approach:

- Store order destination as lat/long.
- Store depot location as lat/long.
- On each telemetry message, compute distance using Haversine formula.
- If distance from truck to destination < 100m → mark order as `ARRIVED`.
- If distance from truck to depot < 200m after delivery accepted → mark truck `JOURNEY_COMPLETE` and return to `IDLE`.

Authentication & Authorization

Amazon Cognito User Pool:

- User sign-up, login, password reset, email verification.
- Integrates with backend via JWT token.

JWT + Backend Validation:

- After Cognito login, client receives JWT with `sub` (user ID), custom claims (`workspace_id`, `roles`).
- API Gateway passes JWT to backend via Authorizer.
- Backend validates JWT, extracts `workspace_id`, passes to RLS in PostgreSQL.

Roles (per workspace):

- `SELLER_MANAGER`: full seller workspace access (accept orders, invite TSPs, review KYC, approve vehicles, view all trucks).
- `TRANSPORT_ADMIN`: upload KYC documents, manage trucks, register vehicles, create tickets, assign trucks to orders.
- `CLIENT`: place orders, accept deliveries.
- `CLIENT_VIEWER`: read-only client access.
- `PLATFORM_ADMIN`: global admin (your company only, handles sensor integration tickets with KYC/approval validation, generates QR codes).

Secrets Management

- AWS Secrets Manager stores:
  - Database credentials.
  - JWT signing key.
  - flespi API key.
  - AWS IoT Core certificates.
  - SES credentials (email).

Data Model

Core Tables (PostgreSQL)

```sql
-- Workspaces (tenant entity)
CREATE TABLE workspaces (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    slug VARCHAR(255) UNIQUE NOT NULL,
    company_type VARCHAR(50),  -- 'SELLER' | 'CLIENT' | 'TRANSPORT'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE
);

-- Users (cross-workspace)
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    first_name VARCHAR(255),
    last_name VARCHAR(255),
    phone_number VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE,
    email_verified BOOLEAN DEFAULT FALSE
);
```

```sql
-- Workspace Memberships (user ↔ workspace + role)
CREATE TABLE workspace_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(50) NOT NULL,  -- SELLER_MANAGER, TRANSPORT_ADMIN, CLIENT,
CLIENT_VIEWER, PLATFORM_ADMIN
    invited_by UUID REFERENCES users(id),
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE,
    UNIQUE(workspace_id, user_id)
);

-- TSP KYC / Compliance Documents (generic document storage)
CREATE TABLE tsp_kyc_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    transport_admin_id UUID NOT NULL REFERENCES users(id),  -- TSP user
    document_type VARCHAR(100),          -- e.g. "DRIVER_LICENSE", "GOVT_ID",
"COMPANY_REG", "SAFETY_CERT"
    document_name VARCHAR(255),          -- free-text label
    file_url VARCHAR(500) NOT NULL,      -- S3 URL or similar
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    reviewed_by UUID REFERENCES users(id), -- Seller Manager
    reviewed_at TIMESTAMP,
    review_status VARCHAR(50) DEFAULT 'PENDING',  -- PENDING, APPROVED,
REJECTED
    review_notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Depots (source locations for sellers)
CREATE TABLE depots (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    address VARCHAR(500),
    latitude DECIMAL(10, 8) NOT NULL,
    longitude DECIMAL(11, 8) NOT NULL,
    timezone VARCHAR(50) DEFAULT 'UTC',
    contact_phone VARCHAR(20),
    contact_email VARCHAR(255),
    geofence_radius_meters INT DEFAULT 200,  -- radius for depot geofence
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE
);

-- Trucks (managed by Transport Admins within seller's workspace)
```

```sql
CREATE TABLE trucks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    transport_admin_id UUID NOT NULL REFERENCES users(id),  -- TSP who
manages this truck
    depot_id UUID NOT NULL REFERENCES depots(id) ON DELETE CASCADE,
    registration_number VARCHAR(50) NOT NULL,
    qr_code_id VARCHAR(255) UNIQUE,  -- Generated after sensor integration
    qr_code_url VARCHAR(500),         -- Generated after sensor integration
    galileosky_device_id VARCHAR(255),  -- NULL until sensor integration
complete
    status VARCHAR(50) DEFAULT 'PENDING_INTEGRATION',
    -- PENDING_INTEGRATION, IDLE, ASSIGNED, EN_ROUTE, AT_DESTINATION,
DELIVERING, RETURNING_TO_DEPOT, MAINTENANCE
    commercial_approval BOOLEAN DEFAULT FALSE,  -- Must be TRUE for sensor
integration
    safety_approval BOOLEAN DEFAULT FALSE,      -- Must be TRUE for sensor
integration
    driver_name VARCHAR(255),
    driver_phone VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE,
    UNIQUE(workspace_id, registration_number)
);

-- Truck Compartments (each compartment capacity defined, fuel type assigned
during order)
-- SIMPLIFIED: directly attached to trucks, no intermediate truck_tanks table
CREATE TABLE truck_compartments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    truck_id UUID NOT NULL REFERENCES trucks(id) ON DELETE CASCADE,
    compartment_index INT NOT NULL,  -- 1, 2, 3, etc.
    capacity_liters DECIMAL(10, 2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(truck_id, compartment_index)
);

-- Fuel Level Sensors (mapped to compartments, configured by Platform Admin)
CREATE TABLE fuel_sensors (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    truck_id UUID NOT NULL REFERENCES trucks(id) ON DELETE CASCADE,
    truck_compartment_id UUID NOT NULL REFERENCES truck_compartments(id),
    sensor_type VARCHAR(50),  -- ANALOG, DIGITAL, MODBUS
    sensor_id_in_device VARCHAR(255),  -- sensor ID from Galileosky device
    calibration_table JSONB,  -- { "0": 0, "100": 50, "255": 100 }
    last_raw_value DECIMAL(10, 2),
    last_liters DECIMAL(10, 2),
    last_updated TIMESTAMP,
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Tickets (for sensor integration and general support)
CREATE TABLE tickets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    ticket_type VARCHAR(50) NOT NULL,  -- SENSOR_INTEGRATION,
MAINTENANCE_REQUEST, TECHNICAL_SUPPORT, etc.
    status VARCHAR(50) DEFAULT 'PENDING',  -- PENDING, ON_HOLD, IN_PROGRESS,
RESOLVED, CLOSED
    priority VARCHAR(50) DEFAULT 'NORMAL',  -- LOW, NORMAL, HIGH, URGENT
    title VARCHAR(255) NOT NULL,
    description TEXT,
    created_by UUID NOT NULL REFERENCES users(id),
    assigned_to UUID REFERENCES users(id),  -- Platform Admin
    related_truck_id UUID REFERENCES trucks(id),  -- for SENSOR_INTEGRATION
tickets
    metadata JSONB,  -- flexible data: sensor details, compartment info, etc.
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    resolved_at TIMESTAMP
);

-- Ticket Comments
CREATE TABLE ticket_comments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ticket_id UUID NOT NULL REFERENCES tickets(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id),
    comment TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Client Companies
CREATE TABLE client_companies (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    contact_phone VARCHAR(20),
    contact_email VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE,
    UNIQUE(workspace_id, name)
);

-- Client Locations (delivery addresses)
CREATE TABLE client_locations (
```

```
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    client_company_id UUID NOT NULL REFERENCES client_companies(id) ON DELETE
CASCADE,
    name VARCHAR(255) NOT NULL,  -- "Main Station", "Branch 2"
    address VARCHAR(500) NOT NULL,
    latitude DECIMAL(10, 8) NOT NULL,
    longitude DECIMAL(11, 8) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE
);

-- Orders
CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    seller_workspace_id UUID NOT NULL REFERENCES workspaces(id),
    client_company_id UUID NOT NULL REFERENCES client_companies(id),
    client_location_id UUID NOT NULL REFERENCES client_locations(id),
    fuel_type VARCHAR(50) NOT NULL,
    requested_volume_liters DECIMAL(10, 2) NOT NULL,
    destination_latitude DECIMAL(10, 8) NOT NULL,
    destination_longitude DECIMAL(11, 8) NOT NULL,
    status VARCHAR(50) DEFAULT 'PLACED',
    -- PLACED → ACCEPTED → ASSIGNED_TO_TSP → TRUCKS_ASSIGNED → EN_ROUTE →
ARRIVED →
    -- DELIVERY_ACCEPTED → OFFLOADING → OFFLOADING_COMPLETE → COMPLETED
    assigned_transport_admin_id UUID REFERENCES users(id),  -- TSP assigned
by seller
    created_by UUID NOT NULL REFERENCES users(id),
    accepted_by UUID REFERENCES users(id),
    notes VARCHAR(500),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Order Truck Assignments (TSP manually assigns trucks and fuel types)
CREATE TABLE order_truck_assignments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    truck_id UUID NOT NULL REFERENCES trucks(id) ON DELETE CASCADE,
    assigned_compartments JSONB,
    -- [{ "compartment_id": "...", "compartment_index": 1, "fuel_type":
"PETROL", "volume": 30, "capacity": 50 }]
    -- TSP assigns fuel_type to each compartment during order assignment
    assigned_volume_liters DECIMAL(10, 2) NOT NULL,
    status VARCHAR(50) DEFAULT 'ASSIGNED',
    -- ASSIGNED → EN_ROUTE → ARRIVED → DELIVERY_ACCEPTED →
    -- OFFLOADING_IN_PROGRESS → OFFLOADING_COMPLETE → JOURNEY_COMPLETE
    pre_delivery_fuel_per_compartment JSONB,  -- { "compartment_1": 100,
```

```
"compartment_2": 80 }
    post_delivery_fuel_per_compartment JSONB,
    actual_delivered_volume DECIMAL(10, 2),
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    arrived_at TIMESTAMP,
    delivery_accepted_at TIMESTAMP,
    offloading_completed_at TIMESTAMP,
    journey_completed_at TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Truck Telemetry (time-series data with per-compartment sensors)
CREATE TABLE truck_telemetry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    truck_id UUID NOT NULL REFERENCES trucks(id) ON DELETE CASCADE,
    timestamp TIMESTAMP NOT NULL,
    latitude DECIMAL(10, 8) NOT NULL,
    longitude DECIMAL(11, 8) NOT NULL,
    speed INT,
    ignition_on BOOLEAN,
    compartment_sensors JSONB,
    -- [
    --   { "compartment_id": "...", "compartment_index": 1, "sensor_id":
"sensor_1", "raw_value": 210, "liters": 35.2 },
    --   { "compartment_id": "...", "compartment_index": 2, "sensor_id":
"sensor_2", "raw_value": 190, "liters": 25.1 }
    -- ]
    -- Note: fuel_type comes from
order_truck_assignments.assigned_compartments during active orders
    total_fuel_liters DECIMAL(10, 2),  -- calculated sum across all
compartments
    device_status VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_truck_timestamp (truck_id, timestamp DESC)
);

-- Delivery Events (timeline log)
CREATE TABLE delivery_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    truck_id UUID REFERENCES trucks(id),
    event_type VARCHAR(100),
    -- ORDER_PLACED, ORDER_ACCEPTED, ORDER_ASSIGNED_TO_TSP, TRUCKS_ASSIGNED,
    -- TRUCK_EN_ROUTE, TRUCK_ARRIVED, DELIVERY_ACCEPTED, OFFLOADING_STARTED,
    -- OFFLOADING_COMPLETED, JOURNEY_COMPLETE, CANCELLED
    event_details JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Notification Log (for tracking sent emails)
```

```sql
CREATE TABLE notification_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID REFERENCES workspaces(id),
    user_id UUID REFERENCES users(id),
    email_address VARCHAR(255) NOT NULL,
    event_type VARCHAR(100),  -- REGISTRATION, TSP_INVITED, KYC_UPLOADED,
KYC_APPROVED, ORDER_PLACED, TICKET_CREATED, etc.
    subject VARCHAR(255),
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) DEFAULT 'SENT'  -- SENT, FAILED, BOUNCED
);
```

RLS Policies (PostgreSQL)

```sql
-- Enable RLS on key tables
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE trucks ENABLE ROW LEVEL SECURITY;
ALTER TABLE depots ENABLE ROW LEVEL SECURITY;
ALTER TABLE tickets ENABLE ROW LEVEL SECURITY;
ALTER TABLE tsp_kyc_documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE order_truck_assignments ENABLE ROW LEVEL SECURITY;
ALTER TABLE truck_telemetry ENABLE ROW LEVEL SECURITY;

-- Example policy: users can see orders only in their workspace(s)
CREATE POLICY orders_workspace_isolation ON orders
    FOR ALL
    USING (workspace_id = current_setting('app.workspace_id')::uuid);

CREATE POLICY trucks_workspace_isolation ON trucks
    FOR ALL
    USING (workspace_id = current_setting('app.workspace_id')::uuid);

CREATE POLICY tickets_workspace_isolation ON tickets
    FOR ALL
    USING (workspace_id = current_setting('app.workspace_id')::uuid);

CREATE POLICY kyc_documents_workspace_isolation ON tsp_kyc_documents
    FOR ALL
    USING (workspace_id = current_setting('app.workspace_id')::uuid);

-- Similar policies for other tables...
```

Core Workflows

4.1 Seller Registration & TSP Invitation

Step 1: Workspace Creation

1. Seller signs up: email, password, first name, last name.

2. Backend creates `users` entry.
3. Sends verification email.
4. After verification, seller creates workspace (name, slug, type: `SELLER`).
5. Backend creates `workspaces` entry, auto-adds seller as `SELLER_MANAGER`.
6. **Email sent**: "Welcome to Fuel Platform! Your workspace is ready."

Step 2: Depot Configuration

1. Manager logs in, adds depot: name, address.
2. Backend geocodes address → lat/long (manual entry or external API).
3. Stores in `depots` table with `workspace_id`, `geofence_radius_meters = 200`.
4. **Email sent**: "Depot added successfully: [Depot Name]"

Step 3: Invite Transportation Service Providers (TSPs)

1. Seller manager navigates to "Manage Transport Providers".
2. Enters TSP email addresses (can add multiple).
3. Backend:
   - Creates pending invitation records.
   - Sends email to each TSP: "You've been invited by [Seller Name] to join their workspace as a Transport Admin."
   - Email includes signup/login link with invitation token.
4. **Email sent (to TSP)**: "Invitation: Join [Seller Name] as Transport Admin"

4.2 TSP Registration & Truck Setup

Step 1: TSP Accepts Invitation

1. TSP clicks link in email.
2. If new user: signs up (email, password, name).
3. If existing user: logs in.
4. Backend:
   - Adds user to seller's workspace as `TRANSPORT_ADMIN`.
   - Marks invitation as accepted.
5. **Email sent (to TSP)**: "You've successfully joined [Seller Name] workspace!"
6. **Email sent (to Seller Manager)**: "[TSP Name] has joined your workspace."

Step 1.1: TSP KYC Documents & Approval

TSP Uploads KYC / Compliance Documents:

- After first login from the invitation link, TSP sees a "Complete KYC" screen if no approved documents exist.
- TSP uploads one or more generic documents (e.g., driver license copies, government IDs, company registration certificates, safety permits, etc.).

    - Each upload includes: `document_type` (e.g., "DRIVER_LICENSE", "GOVT_ID"), `document_name` (custom label), and file.
- Backend:
    - Uploads file to S3 (secure bucket with encryption).
    - Stores file metadata and URLs in `tsp_kyc_documents` with `review_status = 'PENDING'`.
    - Flags the TSP in the workspace as "KYC Pending Review" for seller managers.
- Until at least one KYC document is **approved**, TSP can browse the dashboard but **cannot create sensor integration tickets** or move vehicles to operational status.
- **Email sent (to Seller Manager)**: "[TSP Name] has uploaded KYC documents for review."

Seller Manager Reviews KYC:

- Seller manager opens "Transport Providers → KYC Review" section in dashboard.
- For each TSP, manager:
    - Reviews uploaded documents (views S3 signed URLs).
    - Sets `review_status` to `APPROVED` or `REJECTED`.
    - Optionally adds `review_notes` (e.g., "Missing safety certificate" or "Approved").
- Backend:
    - Updates `tsp_kyc_documents.review_status`, `reviewed_by` (seller manager user ID), and `reviewed_at` timestamp.
    - Derives a TSP-level KYC approval flag: `tsp_kyc_approved = TRUE` if at least one document has `review_status = 'APPROVED'`.
    - This flag is used in business logic to gate ticket creation and sensor integration.
- **Email sent (to TSP)**:
    1. On approval: "Your KYC documents have been approved by [Seller Name]. You can now proceed with truck registration."
    2. On rejection: "Your KYC review requires changes. Please update your documents. Reason: [review_notes]"

Step 2: Truck Registration (Multi-Step Form)

Substep 2a: Basic Info

1. Vehicle registration number.
2. Driver name and phone.
3. Backend validates uniqueness within workspace.
4. Initial **commercial_approval** and **safety_approval** flags are set to `FALSE` by default.
    - These flags can be later updated by the seller manager or an internal review process.

Substep 2b: Compartment Configuration

1. Number of compartments (1, 2, 3, etc.).
2. Capacity per compartment (e.g., Compartment 1: 50L, Compartment 2: 30L, Compartment 3: 20L).
3. Backend creates `truck_compartments` directly for the truck (no intermediate truck_tanks table).
4. **Note: Fuel types NOT assigned at registration time**.

Substep 2c: Create Sensor Integration Ticket

- After completing truck registration, TSP clicks "Request Sensor Integration".
- **Note:** QR code is NOT generated at this point. It will be generated by platform admin after successful sensor integration.
- **Backend validation**:
    1. Checks if `tsp_kyc_approved` is `TRUE` for the TSP in this workspace.
    2. If `FALSE`, returns error: `{ "error": "KYC_NOT_APPROVED", "message": "Please complete and get approval for your KYC documents before requesting sensor integration." }`
    3. TSP is prompted to upload/update KYC documents.
- If KYC approved, backend:
    1. Creates `tickets` entry:
        1. `ticket_type = SENSOR_INTEGRATION`
        2. `status = PENDING`
        3. `created_by = TSP user ID`
        4. `related_truck_id = truck ID`
        5. `title = "Sensor Integration: Truck [Reg Number]"`
        6. `description = "Truck registered with [N] compartments. Awaiting Galileosky sensor configuration."`
        7. `metadata = { "compartments": [...], "truck_details": {...} }`
    2. Updates `trucks.status = PENDING_INTEGRATION`.
    3. Sets `trucks.qr_code_id = NULL` and `trucks.qr_code_url = NULL` (will be generated after integration).
    4. **Email sent (to Platform Admin)**: "New Sensor Integration Request: Truck [Reg] in [Workspace]. KYC Status: Approved."
    5. **Email sent (to TSP)**: "Ticket created. Our team will integrate sensors and generate your QR code shortly."

4.3 Platform Admin: Sensor Integration

Step 1: Admin Receives Ticket

- Platform Admin logs in, views "Pending Tickets" dashboard.
- Sees ticket: "Sensor Integration: Truck [Reg Number]".
- Reviews truck details, compartment configuration.

Step 2: Admin Performs Integration

- Admin accesses flespi dashboard.
- Configures Galileosky device for truck.
- Maps each sensor to corresponding compartment:
    1. Sensor 1 → Compartment 1 (50L capacity)
    2. Sensor 2 → Compartment 2 (30L capacity)
    3. Sensor 3 → Compartment 3 (20L capacity)
- Sets calibration tables per sensor.
- Obtains `galileosky_device_id` from flespi.

Step 3: Admin Validates Prerequisites & Updates Ticket & Truck

Before performing sensor integration, platform admin must verify:

- TSP KYC approval is **TRUE** for the corresponding seller–TSP relationship (at least one `tsp_kyc_documents` record with `review_status = 'APPROVED'`).
- Truck has `commercial_approval = TRUE`.
- Truck has `safety_approval = TRUE`.

If any of these checks fail:

1. Admin keeps `tickets.status = PENDING` (or sets to `ON_HOLD` if desired).
2. Adds an internal note or comment to the ticket indicating which prerequisite failed (e.g., "KYC not approved", "Commercial approval pending", "Safety approval pending").
3. Does **not** configure sensors or generate QR code.
4. **Email sent (to TSP)**: "Sensor integration on hold. Please ensure: KYC approved, commercial approval granted, safety approval granted. Contact [Seller Manager] for details."

If all checks pass, admin proceeds:

- In platform, admin navigates to ticket, clicks "Mark as Resolved".
- Backend:
    - Updates `tickets.status = RESOLVED`, `resolved_at = now()`.
    - Updates `trucks.galileosky_device_id = [device ID]`.
    - Creates `fuel_sensors` entries for each compartment with sensor mappings.
    - **Generates unique `qr_code_id`** for the truck.
    - **Encodes QR code data**: `{ "truck_id": "xxx", "workspace_id": "yyy" }`.
    - **Generates QR PNG image**, uploads to S3.
    - **Stores `trucks.qr_code_url`** with S3 URL.
    - Sets `trucks.status = IDLE` (truck now available for orders).
    - **Email sent (to TSP)**: "Sensor integration complete! Truck [Reg] is ready for orders. QR code available in dashboard."

- **Email sent (to Seller Manager)**: "Truck [Reg] from [TSP Name] is now operational and approved for use."

4.4 Client Registration & Address Management

Step 1: Company Registration

- Client signs up: email, password, company name, phone.
- Backend creates `users` and `client_companies` entry.
- **Email sent**: "Welcome! Your company account is active."

Step 2: Add Delivery Locations

- Client adds location: name, address.
- Backend geocodes address → lat/long.
- Stores in `client_locations` table.
- **Email sent**: "Location added: [Location Name]"

4.5 Order Placement & Assignment Workflow

Step 1: Order Creation (Client)

- Client logs in, places order:
  - Fuel type (DIESEL, PETROL, CNG).
  - Volume (e.g., 100 L).
  - Delivery location.
- Backend:
  1. Creates `orders` entry with status `PLACED`.
  2. Stores destination lat/long.
  3. Creates `delivery_event`: `ORDER_PLACED`.
  4. **Email sent (to seller)**: "New Order! [Order ID] - 100L [Fuel Type] to [Location]"
  5. **Email sent (to client)**: "Order placed successfully. Order ID: [ID]"

Step 2: Order Acceptance & TSP Assignment (Seller)

- Seller manager sees pending order in dashboard.
- Reviews order, clicks "Accept Order".
- **Selects a TSP** from dropdown (shows list of TSPs in workspace).
- Backend:
  - Updates `orders.status = ACCEPTED`.
  - Updates `orders.assigned_transport_admin_id = [TSP user ID]`.
  - Emits event `ORDER_ACCEPTED` and `ORDER_ASSIGNED_TO_TSP`.
  - **Email sent (to client)**: "Your order has been accepted by seller."
  - **Email sent (to TSP)**: "New order assigned to you! Order [ID] - [Volume]L [Fuel Type]. Please assign trucks."

Step 3: Manual Truck Assignment with Fuel Type Selection (TSP)

This is where fuel types are assigned to compartments!

- TSP logs in, sees "Orders Assigned to Me".
- Opens order details: fuel type = PETROL, volume = 100L, destination.
- TSP manually selects trucks:
  - Views available trucks: status = IDLE.
  - Selects **Truck 1** with 3 compartments (50L, 30L, 20L capacities).
  - **For each compartment, TSP assigns fuel type**:
    - Compartment 1 (50L capacity) → assign fuel type: **PETROL** → volume: 50L
    - Compartment 2 (30L capacity) → assign fuel type: **PETROL** → volume: 30L
    - Compartment 3 (20L capacity) → assign fuel type: **PETROL** → volume: 20L
  - Total: 100L PETROL ✓
- **Important**: TSP can assign different fuel types to different compartments of the same truck for the same order (if the order requires mixed fuels), or use only specific compartments.
- Example with mixed fuels (if order was for mixed delivery):
  - Order: 50L PETROL + 30L DIESEL = 80L total
  - Truck 1 compartments:
    - Compartment 1 (50L) → **PETROL** → 50L
    - Compartment 2 (30L) → **DIESEL** → 30L
    - Compartment 3 (20L) → **not used for this order**
- For each truck, TSP also confirms driver assignment.
- Clicks "Assign Trucks".
- Backend:
  - Creates `order_truck_assignments` for each truck with:
    - `assigned_compartments` JSON:

```
[
  {
    "compartment_id": "uuid-1",
    "compartment_index": 1,
    "fuel_type": "PETROL",
    "volume": 50,
    "capacity": 50
  },
```

```
    {
      "compartment_id": "uuid-2",
      "compartment_index": 2,
      "fuel_type": "PETROL",
      "volume": 30,
      "capacity": 30
    },
    {
      "compartment_id": "uuid-3",
      "compartment_index": 3,
      "fuel_type": "PETROL",
      "volume": 20,
      "capacity": 20
    }
  ]
```

    - `status = ASSIGNED`.
  - Updates `trucks.status = ASSIGNED`.
  - Updates `orders.status = TRUCKS_ASSIGNED`.
  - Emits event `TRUCKS_ASSIGNED_TO_ORDER`.
  - **Email sent (to seller)**: "Trucks assigned to Order [ID]: Truck [Reg1]"
  - **Email sent (to client)**: "Truck assigned to your order. Expected delivery: [time]"

4.6 Real-Time Telemetry Processing (Per-Compartment Tracking)

Continuous Telemetry Flow:

1. **Device → flespi:** Galileosky sends GPS + fuel sensor data every 10–30 sec.
    1. Payload includes: `sensor_1_id`, `sensor_1_value`, `sensor_2_id`, `sensor_2_value`, etc.
2. **flespi → AWS IoT Core:** flespi forwards MQTT messages.
3. **IoT Rule → Lambda:** Lambda function `ProcessTruckTelemetry` receives message.
4. **Per-Compartment Fuel Level Calculation:**
    - Lambda:
        - Looks up truck by `galileosky_device_id`.
        - Retrieves `fuel_sensors` mappings: sensor_id → compartment_id.
        - For each sensor in payload:
            1. Maps sensor_id to compartment.
            2. Applies calibration table: raw_value → liters.
            3. Stores per-compartment fuel level.
        - Example:
            - Sensor 1 (210) → Compartment 1 → 35.2 liters

        - Sensor 2 (190) → Compartment 2 → 25.1 liters
        - Sensor 3 (205) → Compartment 3 → 15.2 liters
    - **Total fuel = sum(35.2 + 25.1 + 15.2) = 75.5 liters**
    - **Fuel type per compartment** is determined by looking up active `order_truck_assignments.assigned_compartments`.
5. **Store Telemetry:** Write to PostgreSQL `truck_telemetry` table with `compartment_sensors` JSONB.
6. **Update Truck State:** Update `trucks` latest position, total fuel level.
7. **Geofence Checks:**
    1. If truck assigned to order:
        1. Compute distance from truck position to destination.
        2. If distance < 100m → update `order_truck_assignments.status = ARRIVED`.
        3. Emit event `TRUCK_ARRIVED_AT_DESTINATION`.
        4. **Email sent (to client)**: "Your fuel truck has arrived!"
    2. If truck has completed delivery:
        - Compute distance from truck to depot.
        - If distance < 200m → update `order_truck_assignments.status = JOURNEY_COMPLETE`.
        - Update `trucks.status = IDLE`.
        - Emit event `TRUCK_RETURNED_TO_DEPOT`.
        - **Email sent (to seller & TSP)**: "Truck [Reg] returned to depot. Order [ID] completed."

4.7 Delivery Acceptance & Offloading (QR-Based)

Step 1: Truck Arrives

- Telemetry processing detects truck < 100m from destination.
- Order marked `ARRIVED`.
- Client notified: "Truck has arrived at [Location]!"

Step 2: Client Scans QR

1. Client opens web dashboard on mobile.
2. Navigates to "Accept Delivery".
3. Clicks "Scan QR Code" → camera permission.
4. Scans truck's QR code (generated by platform admin during sensor integration).
5. jsQR decodes → extract `truck_id`.

Step 3: Backend Validation

- Frontend sends: `POST /api/orders/{orderId}/accept-delivery`
  - Payload: `{ "scanned_truck_id": "..." }`
- Backend validates:
  - User is client for this order? ✓
  - Order exists and status = ARRIVED? ✓
  - `scanned_truck_id` has active assignment for this order? ✓
  - Truck within 100m of destination? ✓
- If all pass: accept delivery.
- If fail: return error; client can retry.

Step 4: Delivery Accepted

- Backend:
  - Captures truck's pre-delivery fuel per compartment from latest telemetry.
  - Matches compartment fuel levels with `assigned_compartments` to track which fuel types.
  - Updates `order_truck_assignments.status = DELIVERY_ACCEPTED`.
  - Updates `order_truck_assignments.delivery_accepted_at = now()`.
  - Stores `pre_delivery_fuel_per_compartment` JSON.
  - Emits event `DELIVERY_ACCEPTED_BY_CLIENT`.
  - **Email sent (to TSP)**: "Client accepted delivery for Order [ID]."

Step 5: Offloading

- Backend automatically transitions to `OFFLOADING_IN_PROGRESS` when fuel level changes detected in assigned compartments.
- Or client clicks "Start Offloading" button.
- Emits event `OFFLOADING_STARTED`.

Step 6: Offloading Complete

- Driver physically finishes unloading.
- Confirms via dashboard or backend detects no more fuel outflow.
- Backend:
  - Captures post-delivery fuel per compartment.
  - Calculates delivered volume per compartment: (pre_fuel - post_fuel).
  - Example:
    - Compartment 1 (PETROL): 50L → 5L = **45L delivered**
    - Compartment 2 (PETROL): 30L → 3L = **27L delivered**
    - Compartment 3 (PETROL): 20L → 2L = **18L delivered**
    - **Total delivered: 90L**
  - Updates `order_truck_assignments.status = OFFLOADING_COMPLETE`.
  - Stores `post_delivery_fuel_per_compartment` JSON.
  - Calculates `actual_delivered_volume`.

- Emits event `OFFLOADING_COMPLETED`.
- **Email sent (to client)**: "Fuel delivery complete! [X] liters delivered."

Step 7: Truck Returns to Depot

- Driver drives truck back to depot.
- Telemetry processing detects truck < 200m from depot geofence.
- Backend:
  - Updates `order_truck_assignments.status = JOURNEY_COMPLETE`.
  - Updates `trucks.status = IDLE`.
  - If all trucks for order complete → Updates `orders.status = COMPLETED`.
  - Emits event `TRUCK_RETURNED_TO_DEPOT`.
  - **Email sent (to TSP)**: "Truck [Reg] returned to depot. Ready for new orders."
  - **Email sent (to Seller Manager)**: "Order [ID] fully completed."

4.8 Event List

Event	Trigger	Notification
USER_REGISTERED	User signup & email verified	Welcome email
WORKSPACE_CREATED	Seller creates workspace	Setup guide email
TSP_INVITED	Seller invites TSP via email	To TSP (invitation link)
TSP_JOINED_WORKSPACE	TSP accepts invitation	To TSP + Seller Manager
TSP_KYC_UPLOADED	TSP uploads KYC documents	To Seller Manager
TSP_KYC_APPROVED	Seller Manager approves KYC	To TSP
TSP_KYC_REJECTED	Seller Manager rejects KYC	To TSP
DEPOT_ADDED	Seller adds depot	Confirmation email
TRUCK_REGISTERED	TSP completes truck registration	Confirmation email
TICKET_CREATED	TSP creates sensor integration ticket	To Platform Admin
SENSOR_INTEGRATION_COMPLETE	Platform Admin resolves ticket & generates QR	To TSP + Seller Manager
ORDER_PLACED	Client places order	To Seller Manager + Client
ORDER_ACCEPTED	Seller accepts order	To Client

Event	Trigger	Notification
ORDER_ASSIGNED_TO_TSP	Seller assigns order to TSP	To TSP
TRUCKS_ASSIGNED	TSP manually assigns trucks	To Seller + Client
TRUCK_EN_ROUTE	Truck leaves depot	(In-app update)
TRUCK_ARRIVED_AT_DESTINATION	Truck < 100m from destination	To Client
DELIVERY_ACCEPTED_BY_CLIENT	Client scans QR	To TSP
OFFLOADING_STARTED	Fuel offloading begins	(In-app)
OFFLOADING_COMPLETED	Fuel offloading finishes	To Client
TRUCK_RETURNED_TO_DEPOT	Truck < 200m from depot after delivery	To TSP + Seller
ORDER_COMPLETED	All deliveries done, all trucks returned	To Seller + Client

---

Email Notifications

Email Service Integration

Provider: Amazon SES (Simple Email Service)

- Cost: $0.10 per 1000 emails.
- Transactional emails (invitations, order confirmations, ticket alerts, KYC notifications).
- Configurable sender email (e.g., `noreply@fuelplatform.com`).

Email Events & Templates

1. TSP Invitation

```
Subject: Invitation: Join [SellerName] as Transport Admin
To: tsp_email
Body:
  Hello,

  You've been invited by [SellerName] to join their workspace on Fuel
Platform as a Transport Admin.

  As a Transport Admin, you'll be able to:
  - Register and manage trucks
  - Assign trucks to fuel orders
```

```
  - Monitor delivery status

  [Action Button: Accept Invitation]

  Best regards,
  Fuel Platform Team
```

2. KYC Documents Uploaded (to Seller Manager)

```
Subject: KYC Documents Uploaded - [TSP Name]
To: seller_manager_email
Body:
  Hello [SellerName],

  [TSP Name] has uploaded KYC/compliance documents for review.

  Please review and approve or reject the documents in your dashboard.

  [Action Button: Review KYC Documents]

  Best regards,
  Fuel Platform
```

3. KYC Approved (to TSP)

```
Subject: KYC Documents Approved - [Seller Name]
To: tsp_email
Body:
  Hello [TSPName],

  Great news! Your KYC documents have been approved by [Seller Name].

  You can now proceed with truck registration and sensor integration
requests.

  [Action Button: Register Trucks]

  Best regards,
  Fuel Platform
```

4. KYC Rejected (to TSP)

```
Subject: KYC Review Required - [Seller Name]
To: tsp_email
Body:
  Hello [TSPName],

  Your KYC documents require updates.

  Reason: [review_notes]
```

```
  Please upload updated documents for re-review.

  [Action Button: Update KYC Documents]

  Best regards,
  Fuel Platform
```

5. Ticket Created (to Platform Admin)

```
Subject: New Sensor Integration Request - Truck [RegNumber]
To: platform_admin_email
Body:
  Hello Admin,

  A new sensor integration ticket has been created:

  Ticket ID: [TicketID]
  Workspace: [WorkspaceName]
  Transport Admin: [TSPName]
  Truck Registration: [RegNumber]
  Compartments: [N]
  KYC Status: Approved ✓

  [Action Button: View Ticket]

  Best regards,
  Fuel Platform
```

6. Sensor Integration Complete (to TSP)

```
Subject: Sensor Integration Complete - Truck [RegNumber] Ready!
To: tsp_email
Body:
  Hello [TSPName],

  Great news! Sensor integration for Truck [RegNumber] is complete.

  Your truck is now operational and ready to accept orders.
  QR code has been generated and is available in your dashboard.

  [Action Button: View Truck & Download QR Code]

  Best regards,
  Fuel Platform
```

7. Sensor Integration On Hold (to TSP)

```
Subject: Sensor Integration On Hold - Truck [RegNumber]
To: tsp_email
Body:
  Hello [TSPName],
```

```
  Sensor integration for Truck [RegNumber] is currently on hold.

  Please ensure the following requirements are met:
  - KYC documents approved ✓/✗
  - Commercial approval granted ✓/✗
  - Safety approval granted ✓/✗

  Contact [Seller Manager] for assistance.

  Best regards,
  Fuel Platform
```

8. Order Assigned to TSP

```
Subject: New Order Assigned - [Volume]L [FuelType]
To: tsp_email
Body:
  Hello [TSPName],

  A new order has been assigned to you:

  Order ID: [OrderID]
  Client: [ClientName]
  Fuel Type: [FuelType]
  Volume: [Volume]L
  Delivery Location: [Location]

  Please assign trucks to fulfill this order.

  [Action Button: Assign Trucks]

  Best regards,
  Fuel Platform
```

9. Trucks Assigned (to Client)

```
Subject: Truck Assigned to Order #[OrderID]
To: client_contact_email
Body:
  Hello [ClientName],

  A truck has been assigned to your order:

  Truck: [RegNumber]
  Fuel: [Volume]L [FuelType]

  Expected arrival: [DateTime]

  [Action Button: Track Order]
```

```
  Best regards,
  Fuel Platform
```

10. Delivery Complete (to Client)

```
Subject: Delivery Complete - Order #[OrderID]
To: client_contact_email
Body:
  Hello [ClientName],

  Your fuel delivery has been completed!

  Order ID: [OrderID]
  Fuel Delivered: [Volume]L
  Delivery Time: [DateTime]

  Thank you for using Fuel Platform!

  Best regards,
  Fuel Platform
```

API Design

7.1 Core API Endpoints

Authentication

```
POST /api/auth/signup
  Body: { email, password, firstName, lastName, companyType }
  Response: { userId, workspaceId, token }

POST /api/auth/login
  Body: { email, password }
  Response: { userId, token, workspaces: [...] }

POST /api/auth/accept-invitation
  Body: { invitationToken, password }
  Response: { userId, workspaceId, token }
```

Workspace Management

```
POST /api/workspaces
  Body: { name, slug, companyType }
  Response: { workspace }

GET /api/workspaces
  Response: { workspaces: [...] }

POST /api/workspaces/{workspaceId}/invite-transport-admin
  Body: { email }
```

```
  Response: { invitation }

GET /api/workspaces/{workspaceId}/members
  Response: { members: [...] }
```

KYC Management (TSP & Seller Manager)

```
POST /api/workspaces/{workspaceId}/tsp-kyc-documents
  Body: { documentType, documentName, fileUrl }
  Response: { kycDocument }

GET /api/workspaces/{workspaceId}/tsp-kyc-documents
  Query: { transportAdminId, status, limit, offset }
  Response: { documents: [...], total }

PATCH /api/workspaces/{workspaceId}/tsp-kyc-documents/{documentId}/review
  Body: { reviewStatus, reviewNotes }
  Response: { kycDocument }
```

Depot Management (Seller)

```
POST /api/workspaces/{workspaceId}/depots
  Body: { name, address, latitude, longitude, geofenceRadius }
  Response: { depot }

GET /api/workspaces/{workspaceId}/depots
  Response: { depots: [...] }
```

Truck Management (Transport Admin)

```
POST /api/workspaces/{workspaceId}/trucks
  Body: {
    registrationNumber,
    depotId,
    driverName,
    driverPhone,
    commercialApproval,   // optional, defaults to false
    safetyApproval,       // optional, defaults to false
    compartments: [
      { index: 1, capacity: 50 },
      { index: 2, capacity: 30 }
    ]
  }
  Response: { truck }
  Note: QR code NOT generated at this step

GET /api/workspaces/{workspaceId}/trucks
  Response: { trucks: [...] }

PATCH /api/workspaces/{workspaceId}/trucks/{truckId}/approvals
  Body: { commercialApproval: true, safetyApproval: true }
  Response: { truck }
```

```
GET /api/workspaces/{workspaceId}/trucks/{truckId}/qr-code
  Response: { qrCodeUrl }
  Note: Returns QR code URL after sensor integration complete

GET /api/workspaces/{workspaceId}/trucks/{truckId}/telemetry
  Query: { limit, offset, from, to }
  Response: { telemetry: [...] }
```

Ticket Management (Transport Admin & Platform Admin)

```
POST /api/workspaces/{workspaceId}/tickets
  Body: {
    ticketType: "SENSOR_INTEGRATION",
    title,
    description,
    relatedTruckId,
    metadata
  }
  Response: { ticket }
  Note: If ticketType = "SENSOR_INTEGRATION", backend validates:
    - TSP KYC is approved for this workspace
    - Truck commercial_approval = true
    - Truck safety_approval = true
    Otherwise returns 400 with error: KYC_OR_APPROVALS_PENDING

GET /api/workspaces/{workspaceId}/tickets
  Query: { status, type, limit, offset }
  Response: { tickets: [...], total }

PATCH /api/tickets/{ticketId}/resolve
  Body: { galileoskyDeviceId, sensorMappings: [...] }
  Response: { ticket, qrCodeUrl }
  Note: Backend validates KYC approval + truck approval flags before allowing
resolution.
        Generates QR code as part of resolution process.

POST /api/tickets/{ticketId}/comments
  Body: { comment }
  Response: { comment }
```

Order Management

```
POST /api/workspaces/{workspaceId}/orders
  Body: { fuelType, volumeLiters, clientLocationId }
  Response: { order }

GET /api/workspaces/{workspaceId}/orders
  Query: { status, limit, offset }
  Response: { orders: [...], total }
```

```
POST /api/workspaces/{workspaceId}/orders/{orderId}/accept
  Body: { assignedTransportAdminId }
  Response: { order }

POST /api/workspaces/{workspaceId}/orders/{orderId}/assign-trucks
  Body: {
    truckAssignments: [
      {
        truckId,
        compartments: [
          { compartmentId, compartmentIndex, fuelType: "PETROL", volume: 50,
capacity: 50 },
          { compartmentId, compartmentIndex, fuelType: "DIESEL", volume: 30,
capacity: 30 }
        ]
      }
    ]
  }
  Response: { order, assignments: [...] }

POST /api/workspaces/{workspaceId}/orders/{orderId}/accept-delivery
  Body: { scannedTruckId }
  Response: { success, message }

GET /api/workspaces/{workspaceId}/orders/{orderId}/events
  Response: { events: [...] }
```

Telemetry

```
GET /api/workspaces/{workspaceId}/trucks/{truckId}/current-position
  Response: {
    latitude, longitude, speed, timestamp,
    compartments: [
      { compartmentId, compartmentIndex, currentLiters, capacity, fuelType:
"PETROL" }
    ],
    totalFuel
  }

GET /api/workspaces/{workspaceId}/orders/{orderId}/truck-positions
  Response: { trucks: [...] }
```

Security & Multi-Tenancy

8.1 Tenant Isolation

Database Level:

- All queries filtered by `workspace_id` via RLS.

- Backend sets: `SET app.workspace_id = '<workspace_id>'` on connection.
- RLS policies automatically enforce isolation.

Application Level:

- JWT contains `workspace_id` and `roles`.
- API validates workspace_id matches URL parameter.
- Cross-workspace access denied (except for users in multiple workspaces).

8.2 Authentication

Cognito User Pool:

- Email verification required.
- JWT tokens: `sub`, `email`, `workspace_id`, `roles`.

8.3 Authorization (RBAC)

Role	Actions
SELLER_MANAGER	Accept orders, assign to TSP, invite TSPs, review KYC documents, approve vehicle commercial/safety flags, view all trucks
TRANSPORT_ADMIN	Upload KYC documents, register trucks, create tickets (if KYC approved), assign trucks to orders with fuel type selection
CLIENT	Place orders, accept deliveries
CLIENT_VIEWER	Read-only
PLATFORM_ADMIN	Resolve tickets with KYC/approval validation, sensor integration, QR code generation, all operations

8.4 Data Encryption

- **At Rest**: RDS encryption (KMS), S3 bucket encryption for KYC documents and QR codes.
- **In Transit**: HTTPS, TLS 1.2+.
- **Secrets**: AWS Secrets Manager.
- **KYC Documents**: S3 private bucket with signed URLs (time-limited access).
- **QR Codes**: S3 public bucket (or CloudFront CDN) for easy access.

Deployment & Infrastructure

9.1 AWS Architecture

VPC & Networking:

1. Private subnets for RDS, Fargate.
2. VPC endpoints for S3, Secrets Manager.
3. Multi-AZ for HA.

Compute:

- AWS Fargate (serverless containers).
- 2–4 tasks, auto-scaling on CPU/memory.
- ECS Service with NLB.

Database:

- RDS Aurora Postgres (Serverless v2).
- 0.5–2 ACU auto-scaling.
- Multi-AZ failover.
- RDS Proxy for connection pooling.

IoT:

- AWS IoT Core for MQTT.
- Lambda for telemetry processing (per-compartment logic).
- EventBridge for event routing.

Storage:

- S3 for QR codes (generated post-integration), KYC documents (encrypted, versioned).
- CloudFront for CDN.

Email:

- Amazon SES for transactional emails.

Monitoring:

- CloudWatch logs, metrics, alarms.

9.2 Cost Estimation

Service	Est. Cost/Month
API Gateway	$35
Fargate	$80–120
RDS Aurora Serverless	$60–90
Lambda (IoT)	$20
S3 + CloudFront	$25–50 (includes KYC + QR storage)
SES (email)	$10–15
CloudWatch	$10
Cognito	$0 (free tier)

Service	Est. Cost/Month
Total	**$240–340**

Development Roadmap

Phase 1: MVP (Months 1–3)

Core Features:

- ✓ User registration, login, workspace creation.
- ✓ Seller: depot + TSP invitation.
- ✓ TSP: accept invitation, **KYC document upload**, truck registration, ticket creation.
- ✓ Seller Manager: **KYC document review and approval**, **vehicle approval flags**.
- ✓ **Vehicle commercial and safety approval workflow**.
- ✓ Platform Admin: ticket resolution with **KYC/approval validation**, sensor integration, **QR code generation post-integration**.
- ✓ Client: order placement, address management.
- ✓ Order lifecycle: place → accept → assign to TSP → TSP assigns trucks with fuel type selection → deliver.
- ✓ Per-compartment fuel tracking with dynamic fuel type assignment during orders.
- ✓ flespi + IoT telemetry ingestion.
- ✓ QR-based delivery acceptance.
- ✓ Email notifications on key events (including KYC workflow).
- ✓ Geofence detection (distance-based).
- ✓ Truck journey: depot geofence detection.

Deliverables:

- Backend API (Go, Fargate) with KYC service and ticket prerequisite validation.
- Frontend dashboard (React) with TSP KYC upload, Seller Manager KYC review, Admin ticket management & QR generation views.
- PostgreSQL schema with RLS + KYC document tables + vehicle approval flags.
- S3 storage for KYC documents and QR codes with secure access.
- Email notification system (including KYC events).
- Staging deployment.

Phase 2: Enhancement (Months 4–6)

New Features:

- ✓ SMS/WhatsApp notifications (Twilio).
- ✓ Fuel analytics dashboard (per-compartment trends).
- ✓ Truck maintenance tracking.
- ✓ Order history & reports (CSV export).

- ✓ Pricing & billing model.
- ✓ Advanced KYC document management (expiration tracking, renewal alerts).
- ✓ Bulk QR code printing/download for TSPs.

Phase 3: Advanced (Months 7+)

Features:

- ✓ Route optimization.
- ✓ Predictive analytics (demand, theft detection).
- ✓ Mobile app (iOS/Android) for TSPs and drivers with QR code display.
- ✓ Multi-language support.
- ✓ Advanced security audits.
- ✓ Automated compliance reporting.